Workshop Python Image Analysis

Martijn Wehrens, May 2026

Estimated time: 20 mins presenting + 60 mins exercises + 20 mins discussion

Chapter 6: A very basic introduction into machine learning

> ⚠ Pytorch required
>
> Code in this notebook might need to be run on Google colab or the like, as laptops might not have the right architecture to run the required `pytorch` library.

Outline of this mini-workshop

This chapter introduces quite some concepts, and is sometimes a bit concise. It is intended to be combined with questions or discussions with an instructor that is present.

- Mini lecture about Machine Learning
  - /Users/m.wehrens/Documents/PRESENTATIONS/TEACHING/ML_verybasic.pptx
- Go over the questions in this notebook
  - Don't try to understand all code!

Exercise: draw 4 images

Open FIJI and create a new 12x12 8bit image. Use the paintbrush tool and color picker to draw four images. Draw 2 images depicting one thing, and two images depicting another thing. E.g. draw 2 images of an apple and two images of a pear. Only spend a few minutes on this. (You can also draw something easier, like numbers, symbols, letters, etc.)

Run some code

The code below uses pytorch to set up a very simple neural network. "Real" neural networks have a much more complicated architecture. The purpose here is to be able to follow along what's happening in a neural network.

Understanding, but not the technical details

The code has some explanatory comments, but it takes too much time to explain fully how pytorch or similar neural network libraries (like tensorflow or keras) work. You're welcome to try and understand the code, but don't get lost in it, the main goal is to understand the concepts of machine learning.

```
import torch
from torch import nn

from torch.utils.data import Dataset

import numpy as np

import tifffile as tiff
import matplotlib.pyplot as plt
```

```
# Technical note: ".to(DEVICE)" tells the computer to run this
# neural network on a specific architecture, which enables
# faster calculations.
#
# Common options:
    # CPU: .to("cpu")
    # NVIDIA GPU: .to("cuda") or .to("cuda:0")
    # Apple GPU via Metal: .to("mps")
    # Intel GPU with supported PyTorch build: .to("xpu")

DEVICE = 'mps'
```

```
# This code defines a VERY simple model for 12x12 pixels,
# meant for illustratory purposes.

class VerySimpleNN(nn.Module):
    # Pytorch makes use of classes, which are a way to group
    # data and functions together. A class specifies what functions
    # the class provides, and what data (like parameters) it stores.
```

```
    # An object can then be created to actually load data and
    # call those functions.
    # As an analogy, class definitions are the "blueprint",
    # objects instantiated from the class are the "houses".
    #
    # Here, we define a class (blueprint) for our simple
    # neural network.
    #
    # To learn more about classes, e.g. google for a tutorial on
    # Python classes.

    def __init__(self):
        # __init__ is a standard class method, which is
        # automatically called when an object is created.
        #
        # In this case, it defines how (the layers
        # of) the network should look.

        # technical; calls init of parent class
        super().__init__()

        # Define a flatten function, which in this case
        # can flatten the input (the image) to a 144 element vector
        self.flatten = nn.Flatten()

        # Define a linear layer, which will
        # calculate weights*pixels for 144 element input and 2 output elements
        self.linear = nn.Linear(12*12, 2)

    def forward(self, x):

        # The forward function can be called later to actually
        # generate a prediction.
        # It will use the "components" we defined in __init__.
        # (This is useful when the network is more complicated,
        # and more complex structures or components are defined in __init__, and
        # the forward function will be less complex.)

        # Now actually use the flatten function to convert
        # the 12x12 input to a 1d 144 long vector.
        x = self.flatten(x)
            # technical note:
```

```
            # typically, input will be supplied in batches,
            # so the input shape will be (batchsize, 12, 12)
            # and the output shape will be (batchsize, 144)

        # Now use the linear layer to calculate the 2 element output
        logits = self.linear(x)

        return logits

###
# Now test the neural network

# Instantiate the neural network (create an object from the class)
simpleNN = VerySimpleNN().to(DEVICE)

# Generate a random 12x12 image
test_data = torch.rand(1, 12, 12, device=DEVICE)
print('test_data=',test_data)
# Generate a prediction; calling the object will automatically
# call the forward function (since this is a pytorch class)
# Alternatively, you could also call simpleNN.forward(test_data)
pred = simpleNN(test_data)
pred_numpy = pred.cpu().detach().numpy()
# Print the prediction
print('pred = \"',pred, '\" (pytorch tensor format)')
print('pred_numpy = \"',pred_numpy,'\" (numpy format)')
```

Remarks: Tensors

As you can see the output has the type `tensor`. This is an internal data structure of pytorch, which is similar to a numpy array. The reason we're using these 'special' arrays is (1) that their technical setup allows for calculating the "gradient" (δloss/δw) used to update the weights w and (2) they can be put on GPUs, which can speed up calculations a lot.

Questions

- What role does δloss/δw play in neural networks?
- This questions regards the code under the heading "# Now test the neural network" above. Can you describe in your own terms what
  - The input we give to the network looks like here?

  - What the output of the neural network looks like?

- If we were to provide actual images to this network, do you think it would be able to generate meaningful predictions? Why would it (not)?

Concise answers

- δloss/δw is the gradient, which indicates in which direction the weights should be adjusted to reduce the loss.
- This network expects 12x12 images, the input we give here is just random values in the correct shape. The output is an array corresponding to each of the classes, with a respective score that reflects to what extend the input matched that class. In our example [score_happy, score_sad]. Whichever value is the highest, is the predicted class. The output now shows random numbers since the input as well as the weights had random values.
- No, since the weights are not set during a training setting yet.

```
# Since this network is so simple, we can manually calculate it's outcome

# Now acquire the weights stored in the neural network
# And convert them from tensors to numpy
weights = simpleNN.linear.weight.detach().cpu().numpy()
bias    = simpleNN.linear.bias.detach().cpu().numpy()
    # bias is a constant term that's added to the weighted sum

# Also convert the test data to numpy
test_data_np = test_data.cpu().numpy().flatten()

# Perform the calculation
output_element1 = np.sum(test_data_np * weights[0]) + bias[0]
output_element2 = np.sum(test_data_np * weights[1]) + bias[1]
print(output_element1, ', ', output_element2)
```

Question

- What does the calculation look like that is done inside the neural network, to get from the input elements, to the output elements?

Concise answers

- For each output element, we multiply each input element by its corresponding weight, sum these products together, and then add the bias for that output element.

Exercise: load your images

Adapt the code below to load the images you were asked to draw earlier.

```
# Let's load some potential input and training data
#
# Here, we'll load only the few images you just created.
#
# In reality, training sets are very large.
# For example, the MNIST dataset has 60,000 training images.
# (https://en.wikipedia.org/wiki/MNIST_database)

# Two image paths
img_happy_path = 'images/ML/smile.tif'
img_sad_path   = 'images/ML/sad.tif'
img_sad2_path   = 'images/ML/sad2.tif'

# Load images
img_happy = tiff.imread(img_happy_path)
img_sad   = tiff.imread(img_sad_path)
img_sad2  = tiff.imread(img_sad2_path)

# Show one image
fig, ax = plt.subplots(1,1, figsize=(3/2.54, 3/2.54))
_=ax.imshow(img_happy)
```

Seaborn plotting (can be skipped)

```
# Let's use seaborn to create some more sophisticated plots
import seaborn as sns
def mw_showimg2(img, annotcolor='white',SX=8,SY=8,SF=7,VMIN=0,VMAX=255, CMAP='hot', FMT="d")
    fig, ax = plt.subplots(1,1, figsize=(SX/2.54, SY/2.54))
    _ = sns.heatmap(img, annot=True,
                fmt=FMT,
                cmap=CMAP,
                annot_kws={"size": SF, "color": annotcolor},
                vmin=VMIN, vmax=VMAX,
                linewidths=.5, linecolor=annotcolor,
                ax=ax)
    ax.axis('off')
    ax.collections[0].colorbar.remove()
```

```
    plt.tight_layout()

# Plot the images
mw_showimg2(img_happy, annotcolor='blue')
mw_showimg2(img_sad2, annotcolor='blue')
# Plot some made-up weights
img_randomweights = np.random.uniform(-1, 1, (12, 12))
mw_showimg2(img_randomweights, annotcolor='blue', SF=6,VMIN=-1,VMAX=1.0, FMT=".2f", CMAP='gra
# Plot some made-up output
mw_showimg2(np.array([[13,-4]]), annotcolor='black', SY=4, SF=32,VMIN=-14,VMAX=14,CMAP='hot'
```

Math (can be skipped)

For a prediction y_j, where j is referring to the categories that need predicting (e.g. y_1 high means a "happy" emoji, and a high y_2 value indicates the emoji is likely "sad"), y_j values can be calculated from the ith image pixel x_i as follows (with $w_{i,j}$ being the weights):

$y_j = \sum_i w_{i,j} * x_i$

Defining a data loader

To train a model, pytorch needs to be able to quickly look at a load of data efficiently. In the pytorch workflow, a data class is defined, such that training data can be stored in tensor format, and can be supplied easily to the training algorithm later.

```
class Data_HappySad(Dataset):
    # Again, we use a class, see above for an explanation.
    # (Remember the blueprint/house analogy.)
    #
    # This class is based on the pytorch "Dataset" class, defined
    # in the pytorch library. Data_HappySad is the name we
    # give to this class. Hence the notation "Data_HappySad(Dataset)".

    def __init__(self, targetdevice=DEVICE):

        # Technical: tell the class how to store the data (e.g. on CPU or GPU)
        self.targetdevice = targetdevice

        # Data can be handled in many ways.
        # Here, we'll store some data in the object directly.
        # This can be done by the "self." command, which refers
```

```
        # to the to-be-created object itself.
        #
        # The data needs to be converted to tensor for later use
        # We'll use the images loaded earlier, and store 3 images here
        self.data   = torch.tensor([img_happy, img_sad, img_sad2],
                                   dtype=torch.float32)

        # Now, we supply also the true labels that need to be learned
        self.label  = torch.tensor([0        , 1,       1],
                                   dtype=torch.long)

        # Technical: normalize the images
        self.data = self.data / 255.0

    def __len__(self):
        # Function required by pytorch; tells how many samples are in the dataset

        return len(self.data)

    def __getitem__(self, idx):
        # Function required by pytorch; tells how to get a specific item

        return self.data[idx].to(self.targetdevice), self.label[idx].to(self.targetdevice)
```

```
# Show an image from the dataloader

# Create an object from the class
data_happysad = Data_HappySad()

# We can extract a sample and annotation from that class
# (This will later be done automatically in the training loop)
img, label = data_happysad[0]

# For now, let's convert it to the numpy format for illustration
# purposes
img_np = img.cpu().numpy()

# Plot it
mw_showimg2(img_np, annotcolor='blue', VMIN=0, VMAX=1.0, FMT=".2f", SX=5, SY=5, SF=4)
```

Exercise

- Edit the code above, specifically the subfunction `__init__`, such that it will load the images you created earlier at the start of this notebook.
- Execute the code in the cell directly above, to check if you can make it show your own image.
- This code will show the first image in your dataset, can you also make it show the 2nd and 3rd image?

> Concise answers
>
> - To load your own images, replace the `self.data` and `self.label` appropriately.
> - To show different images, change the index in `data_happysad[0]` to another number.

Dataloader

We're now getting close to actually training this model. We'll need to define a dataloader. In more advanced neural network setups, the DataLoader provides convenient functionalities like shuffling the data, or loading it in parallel from disk. Here, we'll just use it to be able to loop over the data easily, and provide the data in batches of three images.

We'll also initialize the model itself (create an object from the class `VerySimpleNN` defined earlier).

```
# initialize the dataset and dataloader objects
my_data_happysad = Data_HappySad()
my_data_happysad_loader = torch.utils.data.DataLoader(my_data_happysad,
                                                      batch_size=3, shuffle=False)

# initialize the model
my_simple_model  = VerySimpleNN().to(DEVICE)
```

```
# Let's check the data object works as expected
my_data_happysad.__getitem__(0)
```

Question

- The amount of classes and library functions used at this point might be a bit overwhelming. Can you draw a cartoon that depicts the fundamental aspects of the workflow (or otherwise recreate an overview of it)?

Concise answers

- (..)

The actual training

Now we need to train the model by presenting it a picture, calculating the prediction (which might be completely off initially), determine which weight adjustments would improve the prediction (using the loss function), and update the weights accordingly. And repeat that multiple times.

Training a "real" neural network might take presenting the model with many images, for many iterations, and might take hours or days.

Here, our extremely simple model will be trained swiftly.

```
# Create the training loop

# Define a loss function
# This function is defined such that it will have a high value
# if there is a large discrepancy between the predicted and true labels.
# Our aim is to minimize the observed loss during the training.
loss_fn = torch.nn.CrossEntropyLoss()
# Later, we'll determine in which directions to adjust the weights
# (called the gradient), that correspond to a reduction in the loss,
# the optimizer will be used to update (optimize) the weights accordingly.
optimizer = torch.optim.Adam(my_simple_model.parameters(), lr=.01)

# technical; set model to training mode
my_simple_model.train()

# For illustratory purposes, we'll track what happens with the
# gradients and weights during the training. In a more extensive
# model, this isn't possible, as this will be too much data.
gradient_list = [] # only for illustratory purposes
weight_list   = [] # only for illustratory purposes
loss_list     = []
# Now train for 300 iterations (called epochs when 1 iteration covers all data)
for epoch_idx in range(300):

    # This loop is a bit redundant here, since we have only 3 images,
    # in a usual scenario, e.g. with 60.000 images, you would loop
```

```
    # over those images in multiple batches. Here, we'll only
    # have 1 batch, containing three images, so this loop will
    # just iterate once.
    for b_idx, (X,y) in enumerate(my_data_happysad_loader):

        # X will contain a batch of images
        # y will contain the corresponding labels

        # For illustrative purposes, we'll track the weights in the model
        # This isn't part of a usual training loop.
        weights = my_simple_model.linear.weight.detach().cpu().numpy()
        weight_list.append(weights)

        # Now determine the prediction for X based on current weights
        # (Note: the weights are stored inside the model object)
        # (Note: the model is programmed such that it will predict
        # for multiple images at once.)
        pred = my_simple_model(X)

        # Now we'll calculate the loss, or the discrepancy between
        # the predicted and true labels
        loss = loss_fn(pred, y)

        # And based on that calculation, we'll determine the gradients
        # (ie directions in which to adjust the weights to reduce the loss)
        loss.backward()

        # For illustrative purposes, we'll track the gradients
        # This wouldn't be done in a usual training loop.
        gradients = my_simple_model.linear.weight.grad.detach().cpu().numpy()
        gradient_list.append(gradients)
        # Similarly, also save the loss, this is actually also sometimes
        # monitored during "real" training scenarios.
        loss_list.append(loss.item())

        # Then we use the optimizer to apply the gradients
        optimizer.step()
        # And we reset them before the next iteration
        optimizer.zero_grad()

        # Print some information to see what's going on
        print('epoch =', epoch_idx, 'batch =',b_idx,', loss = ', loss.item())#, 'X.shape=', X
```

Questions

- Above you see a training loop, in the printed output below:
  - Why is the loss decreasing?
  - Optional: google what "learning rate" means. This is set by the `lr` parameter. If you like, you can re-initialize the model, adjust `lr` in the cell above, and see what the effect is.
  - What would happen to the loss if you'd train for more iterations?
  - Would you reckon the training has worked? Will the model now recognize something?

Concise answers

- The loss is decreasing since the training is working: a gradient is calculated that shows in which direction each weight should be adjusted to lower the loss. And indeed, this is done each iteration, such that the loss decreases.
- Regarding learning rate; the gradient only gives the direction in which to adjust the weights (how the weights should be adjusted relative to each other), but not by how much. The learning rate scales to what extend the weights are adjusted. If the learning rate is too high, you might "overshoot" the optimal weights, and end up with a higher loss again. If it's too low, training will be very slow.
- If you train for more iterations, the loss will likely decrease further.
- For the original examples, the trianing has likely worked, as the loss decreased a load. However, since the model is so simple, and the training data is so limited, it probably won't be able to recognize anything beyond these specific images.

Loss value over time

```
# Let's plot the loss value over time
_ = plt.plot(np.array(loss_list).squeeze())
plt.xlabel('Iteration (or epoch)'); plt.ylabel('Loss value')
```

Visualization of what happened

The code below visualizes what happened during model training with the weights. Since this model is so simple, we can interpret the weights.

```
# visualize the weights
TO_SHOW_NR = 35
fig, axs = plt.subplots(TO_SHOW_NR//7, 7, figsize=(15/2.54, 15/2.54*(TO_SHOW_NR//7)/7))
# show the weights of weight_list[X][0] in a 5 x 7 grid
for weight_iteration in range(TO_SHOW_NR):
    pos_X = weight_iteration//7
    pos_Y = weight_iteration%7
    _ = axs[pos_X,pos_Y].imshow(weight_list[weight_iteration][0].reshape(12,12), cmap='gray')
    _ = axs[pos_X,pos_Y].axis('off')
plt.suptitle('Weight evolution for "happy" class')

# now the same for the "sad" class
fig, axs = plt.subplots(TO_SHOW_NR//7, 7, figsize=(15/2.54, 15/2.54*(TO_SHOW_NR//7)/7))
# show the weights of weight_list[X][1] in a 5 x 7 grid
for weight_iteration in range(TO_SHOW_NR):
    pos_X = weight_iteration//7
    pos_Y = weight_iteration%7
    _ = axs[pos_X,pos_Y].imshow(weight_list[weight_iteration][1].reshape(12,12), cmap='gray')
    _ = axs[pos_X,pos_Y].axis('off')
plt.suptitle('Weight evolution for "sad" class')
```

Questions

- Update the code `plt.suptitle`s above to reflect the categories that match the labels for the images you created yourself.
- Right to left, top to bottom, we see how the weights are updated over multiple iterations. Why do you think the weights change the way they do?
  - Do you recognize any features in it from your input images?

```
# show the final weights as images
fig, axs = plt.subplots(1,2, figsize=(8/2.54, 4/2.54))
_=axs[0].imshow(weight_list[-1][0].reshape(12,12),cmap='gray')
_=axs[1].imshow(weight_list[-1][1].reshape(12,12),cmap='gray')
_=axs[0].set_title('happy')
_=axs[1].set_title('sad')
_=axs[0].axis('off')
_=axs[1].axis('off')
```

Questions

- Update the labels above according with your own data.
- How do you think this model would perform on a validation data set (as opposed to a training set)?
- Do you think overfitting is a problem for this model?
- In a more complex network architecture, do you think it will be possible to interpret weights in the same way as we did here?

Concise answers

- (..)
- This model will likely perform very poorly on a validation set, since it is overfitted on the few training images it saw. To detect generalized patterns, a more complex model and more training data would be needed.
- So yes, overfitting is a problem. One can test for overfitting by carefully checking performance on a validation set (pictures the model hasn't seen yet), which should be good if the model is not overfitted.
- In a more complex network architecture, it will likely not be possible to interpret the weights in the same way as we did here, since there are many more parameters, and they interact in a more complex way. There are ways to try to interpret the results of complex models, but this is wholly outside our scope, and it is not always possible to fully understand how a complex model makes its decisions.

Proof of the pudding: did it work?

Let's feed the original input images to the model again, and look at the predictions. (In a real scenario, you'd hope the model will also be able to classify unseen data, with our limited training data and limited model, that's not something we can expect.)

```
# apply the model to image 1
X1 = my_data_happysad.__getitem__(0)[0].unsqueeze(0)
pred = my_simple_model(X1)
print('X1=', pred)
# and to image 2
X2 = my_data_happysad.__getitem__(1)[0].unsqueeze(0)
pred = my_simple_model(X2)
print('X2=', pred)
# and to image 3
X3 = my_data_happysad.__getitem__(2)[0].unsqueeze(0)
pred = my_simple_model(X3)
print('X3=', pred)
```

Questions

- Are the predictions correct?
- Try drawing a new image in one of your classes, and let the model predict the class. What do you see?

> Concise answers
>
> - If the location in the array corresponding to the actual class has the highest value, then the prediction is correct.
> - If your old and new images have specific pixels that consistently show different values per class, it might be that the class gets predicted correctly, as this simple model will pick up on those. But as mentioned throughout earlier answers, this model is very simple, and the training data is very limited, so it won't be able to generalize well to new images.

Biological images

- Say you have biological data from multiple conditions and multiple biological replicates. How would you divide this data among the training and validation sets?
- Let's say you use an existing machine learning model (e.g. micro-sam) to segment your data. Do you think you can manually check the segmentation went OK by investigating the original input images and segmentation result?
- Now, let's say you use a machine learning model to quantify a specific phenotype (say 'leaf health' trained on human scoring of leafs) that you expect to change for your conditions.
  - Is it now equally easy as the previous question to check whether the score correctly reflects leaf health?
  - If you use a more complex machine learning model, is it possible to understand how the scoring was determined, or which features the model used to make its decision? What challenges might arise in interpreting the results?
  - Is the interpretability equally important for segmentation and for phenotyping?
  - What are your thoughts on applying ML in this way?
- Say the images used to train the leaf damage also included (in the picture) several groups of insects that contribute to the damage in various degree respectively. Do you see any issue with this training data set?
- How could overfitting impact the performance of your model when applied to new biological images?
- How might differences in image acquisition (e.g. microscope settings, lighting) influence the results of your analysis?
- How can you assess whether your model is generalizing well to unseen data, rather than just memorizing the training set?

Concise answers

My thoughts:

- Ideally, both the training and validation class reflect the same distribution of data, so you would want to have a mix of conditions and biological replicates in both sets. You could for example randomly assign 80% of the data to the training set, and 20% to the validation set, while ensuring that all conditions and replicates are represented in both sets.
- Generally, it's easy to see by eye, even in new images, whether segmentation corresponds to the intended result. Manually drawing segmentations is likely much more work than quickly checking whether the segmentation is correct.
- Phenotype learning:
  - It's not as easy to check whether the score correctly reflects leaf health, since this is a more abstract concept than segmentation. Manually checking all output will likely be difficult, as this is almost as laborious as checking predicted scores.
  - It will be very hard to understand how the model determined its score, in case you have used a more complex model. The model is a "black box".
  - Segmentation usually either has worked or not. How something is recognized is usually not key to understanding your biological question (depending on your topic). On the other hand, understanding the phenotype, and its nuances, is probably much more related to your biological question.
  - I would try to avoid "black box" approaches as solution to biological questions.

Congratulations!

You have reached the end of the machine learning part of this workshop! Let us know if you have any questions, comments or discussion points.

```
img_party=tiff.imread('images/emoji/party.tif')
fig, axs = plt.subplots(1,4, figsize=(12/2.54, 3/2.54))
for ch in range(3):
    _=axs.flatten()[ch].imshow(img_party[:,:,ch])
_=axs.flatten()[3].imshow(img_party)
```

More code for other purposes (can be skipped)

```
# Final weights in different style
print(np.min(weight_list[-1][0]), np.max(weight_list[-1][0]))
mw_showimg2(weight_list[-1][0].reshape((12,12)),
            annotcolor='blue', VMIN=-1, VMAX=1.0, FMT=".2f", CMAP='gray', SF=6)
```

```
# show the first and second gradients
fig, axs = plt.subplots(2,3, figsize=(12/2.54, 6/2.54))
_=axs[0,0].imshow(gradient_list[0][0].reshape(12,12))
_=axs[0,1].imshow(gradient_list[1][0].reshape(12,12))
_=axs[0,2].imshow(gradient_list[2][0].reshape(12,12))
_=axs[1,0].imshow(gradient_list[3][0].reshape(12,12))
_=axs[1,1].imshow(gradient_list[4][0].reshape(12,12))
_=axs[1,2].imshow(gradient_list[5][0].reshape(12,12))
```

```
print(len(gradient_list))
gradient_list[0].shape
gradient_list[4][0].reshape((12,12)).shape
```

```
# Doesn't seem to work ??
# from torchsummary import summary
# summary(my_simple_model, input_size=(12,12))
```

```
# There should be 12x12 + 1 + 12x12 + 1 = 290 model parameters
sum(p.numel() for p in my_simple_model.parameters() if p.requires_grad)
```